3/9/25

Task 5 - Implement Various Searching and Sorting Operations in python programming

Aim:

To implement various Searching and Sorting operations in python programming

5.1 - A company stores employee records in a list of dictionaries where each dictionary contains id, name & department. Write a function find_employee_by_id that takes this list and a target employee ID as arguments and returns the dictionary of the employee with the matching ID or None if no such employee is found.

Algorithm:

1. Input Definition

2. Define the function find_employee_by_id that take two parameters.

a) A list of dictionaries (employees), where each dictionary represents an employee records with keys id, name and department

b) An integer (target_id) representing the employee ID to be searched

3. Iterate through the List: Use a for loop to iterate through each dictionary in the employees list

4. Check for Matching ID: Within the loop, check if the filed of the current dictionary matches the target_id

5. Return Matching Record: If the match is found, return the current dictionary

6. Handle No Match: If the loop completes

Output:

{'id': 2, 'name': 'Bob', 'department': 'Engineering'}

without finding a match, return None.

Program:

```
def find_employee_by_id(employees, target_id):
    for employee in employees:
        if employee['id'] == target_id:
            return employee
    return None
employees = [ {'id': 1, 'name': 'Alice', 'department': 'HR'},
    {'id': 2, 'name': 'Bob', 'department': 'Engineering'},
    {'id': 3, 'name': 'Charlie', 'department': 'Sales'},
]
print (find_employee_by_id (employees, 2))
```

5.2 - Grade Management System

Algorithm:

1. Initialization:
   - Get the length of the students list store it in n
2. Outer Loop:
   - Iterate from i=0 to n-1. This loop represents the number of passes through the list
3. Track Swaps:
   - Initialize a boolean variable swapped to false. This variable will track if any swaps are made in the current pass
4. Inner Loop:
   - Iterate from j=0 to n-i-2. This loop compares adjacent elements in the list and performs swaps if necessary
5. Compare and Swap:
   - for each pair of adjacent elements
     - Compare their score values
     - If students[j]['score'] > students[j+1]['score'], swap the two elements. To indicate the swap was made set it as

Output:

Before Sorting:

{'name': 'Alice', 'score': 88}

{'name': 'Bob', 'score': 95}

{'name': 'Charlie', 'score': 75}

{'name': 'Diana', 'score': 85}

After Sorting:

{'name': 'Charlie', 'score': 75}

{'name': 'Diana', 'score': 85}

{'name': 'Alice', 'score': 88}

{'name': 'Bob', 'score': 95}

True.

6. Early Termination:

- After each pass of the inner loop, check if swapped is false. If no swaps were made during the pass, the list is already sorted and you can break out of the outer loop early

7. Completion:

- The function modifies the students list in place, sorting it by score.

Program:

```
def bubble_sort_scores(students):
    n = len(students)
    for i in range(n):
        swapped = false
        for j in range(0, n-i-1):
            if students[j]['score'] > students[j+1]['score']:
                students[j], students[j+1] = students
                                              [j+1], students[j]
                swapped = True
        if not swapped:
            break
students = [ {'name': 'Alice', 'score': 88}, {'name': 'Bob', 'score': 95},
        {'name': 'Charlie', 'score': 75}, {'name': 'Diana', 'score': 85}
]
print ("Before sorting:")
for student in students:
    print (student)
bubble_sort_scores(students)
print ("\n After sorting:")
for student in students:
    print (student)
```

VEL TECH	
EX NO.	5
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	15

Result:

Thus the Program for various Searching and Sorting Operations is executed and verified successfully.